

Higher-MI Samajhne Ke Liye

O-Information se Grokking Predictor Tak — Zero Gap Walkthrough

Jonathan ke liye

October 3, 2026

Contents

1	Roadmap — Hum Kahan Ja Rahe Hain	2
2	Entropy Ka Recap — “Kitna Phela Hua Hai”	2
3	Do Variables Judna: Joint Aur Conditional Entropy	2
4	Mutual Information: Do Variables Kitna Share Karte Hain	3
5	Teesra Variable: Conditional Mutual Information	3
6	O-Information: Redundancy Vs Synergy	3
7	Teen Se Zyada Nazariye: Permutation Averaging	4
8	Continuous Numbers Ki Problem: Histogram Kaam Nahi Karta	4
9	KSG Estimator: Nearest Neighbor Se Mutual Information Nikalna	5
10	Conditional MI Bhi KSG Se: Chaar Spaces Ka Trick	5
11	Bug 1: Strict Vs Inclusive Neighbor Counting	5
12	Bug 2: Test Set Chupke Se Badal Gaya Tha	6
13	Ek Checkpoint Se Teen Random Variables Tak	6
14	Major Adaptation: Paper Vs Project — Population Vs Temporal	7
15	Event Rule: Global Maximum Kyun (Minimum Nahi)	7
16	Tumhare Project Me Kya Actually Hua — Raw Curves	8
17	Frozen Protocol Verdict	8
18	Implementation Ki Duvidha Aur Problems — Summary	9
19	Bada Picture: Thesis Ke Liye Matlab	9

1 Roadmap — Hum Kahan Ja Rahe Hain

Tumhara sawaal tha: Higher-MI kaise kaam karta hai, aur isko implement karte waqt kya duvidha aur problems aayi. Ye predictor WeightPCA se kaafi alag family ka hai — WeightPCA ek matrix ki singular values dekhta tha (linear algebra), Higher-MI information theory se aata hai: ye naapta hai ki teen quantities ek-doesre ke baare me kitni “information share” karti hain.

Plan: pehle entropy ka chhota recap (WeightPCA document me detail se dekha tha), fir do variables judne se mutual information banayenge, fir teesra variable add karke conditional mutual information aur O-information tak pahunchenge — O-information hi is predictor ka core signal hai. Fir dekhenge ki continuous numbers (jaise model ke logits) par ye formulas seedha kyun nahi lagte, aur KSG estimator kaise is problem ko solve karta hai. Uske baad do implementation bugs dekhenge jo is project me aaye, fir dekhenge Higher-MI tumhare Nanda-Unified setup me kaise adapt hua, aur aakhir me actual result aur verdict.

Har section pichle section par khada hai, jaisa pichli baar tha.

2 Entropy Ka Recap — “Kitna Phela Hua Hai”

WeightPCA walkthrough me entropy dekhi thi: agar $p_1, \dots, p_R$ non-negative numbers hain jo sum hoke 1 banate hain (ek probability distribution), to

$$H(p) = - \sum_{i=1}^R p_i \log p_i.$$

Jitna distribution spread hua (sab p_i barabar-barabar), entropy utni zyada; jitna concentrated (ek p_i dominant), entropy utni kam.

Yahan entropy ko ek random variable X ki “uncertainty” ke roop me padhenge: $H(X)$ batata hai, agar tumhe X ki value dekhne se pehle poochha jaaye “ye kya nikalega”, to average me kitna “surprise” hoga. Agar X hamesha ek hi value leta hai, $H(X) = 0$ — koi surprise nahi. Agar X saari values equally likely leta hai, $H(X)$ maximum hai — sabse zyada surprise.

3 Do Variables Judna: Joint Aur Conditional Entropy

Ab do random variables X aur Y ek saath lo. Inki joint distribution $p(x, y)$ hoti hai — har pair (x, y) ki probability. Isi par entropy formula lagao:

$$H(X, Y) = - \sum_{x, y} p(x, y) \log p(x, y).$$

Ye “dono ko ek saath dekhne par total uncertainty” hai.

Ab sawaal: agar tumhe Y ki value pehle hi pata chal jaaye, to X me kitni uncertainty *bachi* rehti hai? Isko conditional entropy kehte hain:

$$H(X | Y) = H(X, Y) - H(Y).$$

Intuition: total uncertainty $H(X, Y)$ me se, Y akela jitni uncertainty leke aata hai ($H(Y)$), wo nikaal do — jo bacha, wahi X ki “extra” uncertainty hai jo Y jaanne ke baad bhi bachi hui hai.

Chhota worked example: $X, Y \in \{0, 1\}$, aur maan lo (X, Y) in chaar combinations me se equally (probability 1/4 har ek) aata hai: $(0, 0), (0, 1), (1, 0), (1, 1)$ — yaani X aur Y poori tarah independent hain. Yahan $H(X, Y) = \log 4$, $H(Y) = \log 2$, isliye

$$H(X | Y) = \log 4 - \log 2 = \log 2 = H(X).$$

Matlab Y jaanne se X ki uncertainty bilkul kam nahi hui — Y ne X ke baare me kuch bataya hi nahi. Ye independent variables ka signature hai, agle section me kaam aayega.

4 Mutual Information: Do Variables Kitna Share Karte Hain

Pichle example me $H(X | Y) = H(X)$ tha — Y se kuch fayda nahi hua. Ab ulta case sochte hain: agar $Y = X$ bilkul (dono hamesha same value lete hain), to Y jaante hi X pura pata chal jaata hai, matlab $H(X | Y) = 0$.

In dono extremes ke beech ka gap hi naapna hai: Y jaanne se X ki uncertainty *kitni kam hui*. Isi ko mutual information kehte hain:

$$I(X; Y) = H(X) - H(X | Y).$$

Pehle (independent) example me $I(X; Y) = H(X) - H(X) = 0$. Dusre (identical) example me $I(X; Y) = H(X) - 0 = H(X)$ — maximum possible.

Do zaroori properties (bina proof ke le rahe hain, standard result hai): $I(X; Y) \geq 0$ hamesha, aur $I(X; Y) = 0$ *sirf tabhi* jab X, Y bilkul independent hon. Aur ye symmetric hai: $I(X; Y) = I(Y; X)$ — iska matlab ek formula alag se bhi likha ja sakta hai: $I(X; Y) = H(X) + H(Y) - H(X, Y)$, jo upar wali definition ke algebra se hi nikalta hai.

5 Teesra Variable: Conditional Mutual Information

Ab ek teesra variable Z add karte hain. Sawaal badal jaata hai: agar humein Z *pehle se hi* pata ho, tab X aur Y kitna share karte hain? Isko conditional mutual information kehte hain:

$$I(X; Y | Z) = H(X | Z) - H(X | Y, Z).$$

Bilkul Section 4 jaisi hi definition hai, bas ab har jagah “ Z ko already pata maan kar” naapa ja raha hai. Ye bhi hamesha ≥ 0 hota hai.

Yahan ek zaroori, thoda counter-intuitive point hai jo agle section ka poora motivation hai: Z ko jaanne se X, Y ka shared information *kam ho sakta hai, barabar reh sakta hai, ya badh bhi sakta hai* — teeno possible hain. Agla section isi teesre (badhne wale) case ko explore karta hai, kyunki wahi Higher-MI predictor ka poora point hai.

6 O-Information: Redundancy Vs Synergy

Compare karo: Z jaanne se pehle X, Y kitna share karte the ($I(X; Y)$), aur Z jaanne ke baad kitna share karte hain ($I(X; Y | Z)$). Inka difference hi O-information hai:

$$\Omega = I(X; Y) - I(X; Y | Z).$$

Do sign cases ka matlab bilkul alag hai:

Redundant case ($\Omega > 0$): Z jaanne se X, Y ka shared information *kam* ho gaya — matlab Z khud hi wo information repeat kar raha tha jo X, Y pehle se share kar rahe the. Z “naya” kuch nahi laaya, sirf dohra raha tha.

Worked example: $X \in \{0, 1\}$ fair coin, aur $Y = X, Z = X$ — teeno hamesha same value lete hain. Pehle: $I(X; Y) = H(X) = \log 2 \approx 0.693$ (natural log, Section 4 ka identical-variable extreme

case). Ab Z condition karo: $Z = X$ maloom hote hi X pura pata chal jaata hai ($H(X | Z) = 0$), isliye $I(X; Y | Z) = 0$ bhi (kuch bachta hi nahi share karne ko). Isliye

$$\Omega = 0.693 - 0 = +0.693 > 0 \quad (\text{redundant}).$$

Synergistic case ($\Omega < 0$): Z jaanne se X, Y ka shared information *badh* gaya — matlab X, Y akele kuch share nahi karte the, par Z ke context me unka rishta dikhne lagta hai. Ye classic XOR example se sabse saaf dikhta hai.

Worked example: $X, Y \in \{0, 1\}$ dono independent fair coins, aur $Z = X \oplus Y$ (XOR — $Z = 1$ agar X, Y alag hain, $Z = 0$ agar same hain). Pehle: X, Y independent hain (by construction), isliye $I(X; Y) = 0$ (Section 4 ka independent-variable extreme case). Ab Z condition karo: agar Z pata hai, to X jaante hi Y turant pata chal jaata hai ($Y = X \oplus Z$, deterministic), isliye $I(X; Y | Z) = H(X | Z) = H(X) = \log 2 \approx 0.693$ (Z jaanne se X ki apni distribution badalti nahi, wo abhi bhi fair coin hai). Isliye

$$\Omega = 0 - 0.693 = -0.693 < 0 \quad (\text{synergistic}).$$

Ye dono examples hi is predictor ki poori hypothesis ka dil hain: Ω ka sign batata hai ki teen variables “redundant” tarah behave kar rahe hain ya “synergistic” tarah. Pomarico et al. (2025, arXiv:2507.23346) ka paper suggest karta hai ki neural network training ke dauraan ye balance shift hota hai, aur grokking ke aas-paas ek dikhne-layak badlaav hota hai.

7 Teen Se Zyada Nazariye: Permutation Averaging

$\Omega = I(X; Y) - I(X; Y | Z)$ me teen variables ka role symmetric nahi dikhta — Z “conditioning” role me hai, X, Y “shared” role me hain. Par asal me koi bhi variable Z ka role le sakta hai: tumhare paas teen choices hain — Z ban sakta hai X, Y , ya teesra variable khud.

Paper ka Eq. 13 teeno choices (aur unke 6 orderings, kyunki X, Y ka order bhi permute kiya ja sakta hai) ka average leta hai. Par ek zaroori simplification: $I(X; Y) = I(Y; X)$ hamesha (Section 4 ka symmetry fact), aur $I(X; Y | Z) = I(Y; X | Z)$ bhi isi tarah symmetric hai. Isliye X, Y ko swap karne se Ω badalta hi nahi — 6 permutations me se har pair repeat hoti hai. Matlab asal me sirf 3 distinct values hain (“kaun conditioning role me hai” ke 3 choices), aur 6-permutation average sirf inhi 3 values ka plain average hai, do-do baar gin ke. Tumhare code (`o_information_three_way`) yahi karta hai.

8 Continuous Numbers Ki Problem: Histogram Kaam Nahi Karta

Ab tak X, Y, Z discrete the (coin flips, finite values) — $p(x, y, z)$ seedha count karke nikal sakte the. Tumhare project me X, Y, Z model ke logits hain — real numbers, continuous. Ek seedha tarika hoga: values ko bins me daal do (histogram banao), fir upar wale discrete formulas lagao.

Problem ye hai ki histogram ka result bin-size ki choice par bahut depend karta hai — chhote bins se noisy estimate, bade bins se detail kho jaata hai, aur high dimensions me (yahan 2–3 variables ek saath) bins ki sankhya exponentially badh jaati hai jabki data points utne nahi badhte (“curse of dimensionality”). Isliye ek aisa estimator chahiye jo bina binning ke seedha continuous data se kaam kare — yahi KSG estimator karta hai.

9 KSG Estimator: Nearest Neighbor Se Mutual Information Nikalna

KSG (Kraskov–Stögbauer–Grassberger) estimator ka idea: binning ki jagah, har data point ke *nearest neighbors* ki doori dekho — wo doori khud hi batati hai us jagah data kitna “dense” hai, bina kisi bin-size choose kiye.

Mutual information ke liye (paper ka Eq. A7 jaisa): N data points lo (joint (x, y) pairs), aur ek fixed k choose karo (tumhare code me $k = 4$ — ye assumption hai, paper is adapted setup ke liye explicitly nahi deta). Har point i ke liye:

1. Joint (x, y) space me point i ka k -vaan nearest neighbor dhoondo (max-norm metric me), aur uski doori ko ε_i bolo.
2. $n_x(i)$ = kitne *doosre* points ka x -coordinate, point i ke x -coordinate se ε_i se *kam* doori par hai (sirf x -axis par doori, poore joint space me nahi).
3. Waisa hi $n_y(i)$, y -coordinate ke liye.

Fir estimator hai:

$$I(X; Y) \approx \psi(k) - \langle \psi(n_x + 1) + \psi(n_y + 1) \rangle + \psi(N),$$

jahan ψ digamma function hai (log ka ek smooth cousin, iska exact formula yahan zaroori nahi) aur $\langle \cdot \rangle$ saare N points par average hai.

Intuition (bina heavy math ke): agar X, Y independent hain, to joint space me point i ke paas jo k neighbors hain, unka x - aur y -coordinates *bhi* roughly utne hi “paas” honge jitna akele x -axis ya y -axis par expect hota — n_x, n_y roughly k ke barabar aayenge. Agar X, Y dependent hain (saath-saath badalte hain), to joint space me point i ke paas neighbors honge jo *marginal* x - ya y -axis par bhi unexpectedly paas honge — n_x, n_y zyada bade aayenge. Formula isi extra-closeness ko numbers me convert karta hai.

10 Conditional MI Bhi KSG Se: Chaar Spaces Ka Trick

$I(X; Y | Z)$ ke liye wahi idea extend hota hai, bas ab ε_i *teeno* variables (x, y, z) ke joint space me nikalta hai, aur teen alag marginal counts chahiye: $n_{xz}(i)$ ((x, z) pair space me, doori ε_i se kam), $n_{yz}(i)$ ((y, z) space me), aur $n_z(i)$ (sirf z -axis par). Estimator (paper ka Eq. A9 jaisa):

$$I(X; Y | Z) \approx \psi(k) - \langle \psi(n_{xz} + 1) + \psi(n_{yz} + 1) - \psi(n_z + 1) \rangle.$$

Dhyan do: ab $\psi(N)$ ki jagah $\psi(n_z + 1)$ ka average aa gaya hai, kyunki ab reference “poore dataset” se nahi, balki “ Z ke around jo local neighborhood hai” se liya ja raha hai — yahi “conditional” hona hai.

11 Bug 1: Strict Vs Inclusive Neighbor Counting

KSG formula ki definition maangti hai: $n_x(i)$ un points ka count jo ε_i se **strictly kam** (“ $<$ ”) doori par hain — ε_i ke barabar doori wale count *nahi* hote.

Project me neighbor-counting ke liye `scipy's cKDTree.query_ball_point` use hua, jo by default **inclusive** (“ $\leq$ ”) counting karta hai — ε_i ke barabar doori wale points bhi count kar leta hai.

Chhota worked example jisse farak dikhe: maan lo query point i se teen points ki doori hai 0.5, 0.9, aur *bilkul* 1.2 (tie, kisi integer-valued ya repeated-structure data me ye aam hai). Agar $\varepsilon_i = 1.2$ hai (yaani k -vaan neighbor exactly 1.2 doori par hai), to:

- Sahi (strict, “ < 1.2 ”) count: sirf 0.5 aur 0.9 wale do points — $n = 2$.

- scipy ka default (inclusive, “ ≤ 1.2 ”): teeno points, 1.2 wala bhi shaamil — $n = 3$.

Ek extra point count ho jaana chhota sa farak lagta hai, par ye systematically n_x, n_y (ya n_{xz}, n_{yz}, n_z) ko thoda zyada bada deta hai har query point par, jo $\psi(n + 1)$ ko bhi zyada karta hai, jo final I estimate ko *neeche* bias kar deta hai (kyunki $\psi(n + 1)$ formula me minus sign ke saath hai) — chhota par systematic bias, poori training history me consistently.

Fix: ε_i ki jagah query radius `np.nextafter( $\varepsilon_i, 0$ )` use kiya — yaani “ ε_i se chhota sabse bada float number”. Isse scipy ka inclusive ($\leq$) query effectively strict ($<$) ban jaata hai, bina khud koi custom neighbor-search likhe. Fix ko ek dense (brute-force, sab pairs check karne wala) reference implementation ke against bit-for-bit verify kiya gaya.

Is fix ka ek zaroori side-effect bhi tha: `query_ball_point` tree-based search use karta hai, jo dense brute-force se kaafi fast hai. Naive dense counting me roughly 19 seconds/checkpoint lagte the; tree-based query ke saath (fix ke baad) sirf 0.055 seconds/checkpoint laga neighbor-counting ke liye akele. Poora per-checkpoint pipeline (chaaron signals + checkpoint load + model forward pass + file I/O milake) 0.67–0.82 seconds/checkpoint aaya. Ye fix sirf correctness ke liye nahi tha — iske bina poora 5×401 -checkpoint benchmark (~ 27 minute) practically chalana hi mushkil hota (19s/checkpoint $\times 2005$ checkpoints ≈ 10.5 ghante).

12 Bug 2: Test Set Chupke Se Badal Gaya Tha

Ye bug KSG se bilkul alag hai — ye data-pipeline ka mistake hai, formula ka nahi, aur ye crash nahi karta, isliye pakadna mushkil tha.

Training ke dauraan, model ka train/test split banta hai `torch.manual_seed(seed)` call ke *turant baad* — taaki wahi seed se, wahi split reproduce ho sake baad me bhi.

`run_nanda_benchmark.py` ka `recompute_from_checkpoints()` function (jo saved checkpoints se predictors recompute karta hai, bina retraining ke) apna `test_loader` seedha bana leta tha, **bina pehle seed lagaye**. Matlab ye function jo test split bana raha tha, wahi split **nahi** tha jo live training ke waqt actually use hua tha — ek alag, random split.

Iska asar: Higher-MI ke logits (correct/runner-up/rest-mean) galat test examples par compute ho rahe hote — numbers aate, crash nahi hota, bas wo galat set ke against the. Ye isliye khatarnaak hai ki koi error message nahi aata, result bas *thoda* galat hota, aur easily unnoticed reh sakta hai.

Fix: `_checkpoint_predictor_higher_mi` function apna khud ka `test_loader` banata hai, `torch.manual_seed()` aur `np.random.seed(seed)` explicitly call karke, phir `get_data_loaders()` — bilkul training ke waqt jaisa order. Ye koi naya pattern nahi hai — `_checkpoint_predictor_spectral` aur `_checkpoint_predictor_a` pehle se hi yahi karte the apne train-loader ke liye. Higher-MI isi precedent ko follow karta hai.

13 Ek Checkpoint Se Teen Random Variables Tak

Section 6 ke X, Y, Z ab tumhare model ke logits se banane hain. Tumhara task $(a + b) \bmod 113$ hai, isliye model har input par 113 classes par ek logit value deta hai (“=” token ka 114-vaan column pehle hi drop kar diya jaata hai, kyunki wo kabhi train hi nahi hota).

Teaching ke liye chhota $p = 5$ example lete hain (project me asal $p = 113$ hai, same formula). Maan lo ek example ka true answer class-index 2 hai, aur model ke 5 logits hain [1.0, 5.0, 8.0, 2.0, 4.0] (0-indexed). Teen numbers nikalte hain:

- **correct** = true class ka apna logit = $\text{logits}[2] = 8.0$.
- **runner-up** = baaki 4 classes me sabse bada = $\max(1.0, 5.0, 2.0, 4.0) = 5.0$.

- **rest-mean** = baaki bache 3 classes ka average (correct aur runner-up dono hata kar) = $\frac{1.0+2.0+4.0}{3} = \frac{7}{3} \approx 2.333$.

Ye teeno (correct, runner-up, rest-mean) ek example ke liye teen numbers hain. Poore test set (saare examples) par ye compute karo — har example ek “sample” deta hai, aur poore test set ke saare correct-values milke ek random variable F_0 ki realizations ban jaate hain (waise hi F_1 = runner-up values, F_2 = rest-mean values). Ek checkpoint par, yahi F_0, F_1, F_2 ban kar Section 6–10 ke KSG + O-information pipeline me jaate hain, aur ek number $\Omega_{\text{population}}$ deta hai — usi checkpoint ka.

14 Major Adaptation: Paper Vs Project — Population Vs Temporal

Pomarico et al. ka paper apni original setting me X, Y, Z ko *time series* ki tarah treat karta hai — same quantity, baar-baar alag time steps par nahi gayi (unki tensor-network setup me). Tumhare project me aisa koi natural “repeated-over-time single example” axis nahi hai jo poore training run me ek saath chale.

Isliye ek **MAJOR ADAPTATION** ki gayi (confirmed 2026-10-03): ek checkpoint par, F_0, F_1, F_2 ke samples *time se nahi, test-set ke saare examples se* aate hain — “population” variant. Ye hi PRIMARY signal hai.

Is adaptation ka ek seedha risk hai, jo implementation shuru karne se pehle hi flag kiya gaya tha: agar kuch test examples *hamesha hi* aasaan hon (high correct logit har checkpoint par) aur kuch hamesha hard, to correct aur runner-up logits “dependent” dikh sakte hain sirf isliye ki wo ek shared, static “example difficulty” confound share karte hain — grokking se related kisi mechanism ki wajah se nahi. Agar aisa hai, to $\Omega_{\text{population}}(t)$ shayad training ke poore time me hamesha redundant dikhega, bina kisi genuine phase-transition signal ke — predictor “hamesha PASS jaisa dikhega” ya “kabhi meaningful transition nahi dikhayega”, dono galat.

Isi concern ko directly test karne ke liye, ek **EXPLORATORY CONTROL** add kiya gaya: “temporal” variant. Yahan F_0, F_1, F_2 ke samples ek *sliding window* ($W = 100$ checkpoints, stride = 10) ke andar, per-checkpoint *mean* logit values se aate hain — ye paper ki literal time-series reading ke zyada paas hai. Agar population aur temporal signals bilkul same shape dikhaate, to confound-risk sach saabit hota (dono ek hi cheez naap rahe hote). Agar alag dikhte, to population signal kam se kam ek genuinely alag quantity naap raha hai — confound poori tarah khaarij nahi hota, par ye ek zaroori sanity check hai.

15 Event Rule: Global Maximum Kyun (Minimum Nahi)

WeightPCA me event rule tha “global minimum” — kyunki hypothesis thi rank *collapse* (neech jaana). Higher-MI me hypothesis ulti hai: training ke shuru me (memorization phase) logits messy, entangled, possibly synergistic ho sakte hain; generalization aane par ek cleaner, zyada redundant pattern ban sakta hai (model teeno quantities ko consistent tarike se treat karne lagta hai). Isliye PRIMARY event rule hai **global maximum** — wo epoch jahan $\Omega_{\text{population}}(t)$ apni sabse badi (sabse zyada redundant) value par pahunchta hai, poori training history me. Ye function (`find_global_maximum_epoch`) hamesha defined hai, kabhi “None” nahi aata.

Do exploratory rules bhi rakhe gaye, scoring me nahi: **zero-crossing epoch** — wo epoch jahan $\Omega_{\text{population}}(t)$ pehli baar negative (synergistic) se positive (redundant) paar karta hai, ek natural

“phase-switch” candidate; aur **temporal-control maximum** — Section 14 wale control signal ka apna global maximum.

16 Tumhare Project Me Kya Actually Hua — Raw Curves

Poore 5 seeds $\times$ 401 checkpoints par (saved checkpoints se recompute, koi retraining nahi), raw curve pehle dekhi gayi, scoring se pehle — ye tumhari apni stated practice thi: “raw $\Omega(t)$ pehle dekho, event epochs par trust karne se pehle.”

Jo dikha: $\Omega_{\text{population}}(t)$ **poori training history me oscillatory aur quasi-periodic** hai — ek clean single-peak curve nahi, balki baar-baar 0 ke paas se upar spike karta hai aur wapas neeche aata hai, poori training me, aur spike ki size training ke saath generally badhti jaati hai (seed 4 me starting value 0.037 se final value 1.80 tak, roughly $48\times$ badhaav).

Seed 0 ka population-vs-temporal comparison bhi dikhaya gaya: dono qualitatively alag dikhe — population baar-baar spike karta raha, temporal zyada smooth tha, aur epoch $\sim 9\text{--}10,000$ (grok se thoda pehle) ke aas-paas ek minimum dikhaya, fir uske baad badha. Ye alag-alag shapes confound ke poore sach hone ke against hain (agar confound poora sach hota, dono curves same dikhti), par is baat ko nahi badalta ki dono me se koi bhi clean pre-grok signal nahi hai.

Per-seed numbers ($\Omega_{\text{population}}$ ka maximum kis epoch par aaya, grok epoch ke comparison me):

Seed	Grok epoch	$\Omega_{\text{population}}$ max epoch	Ratio (max / grok)
0	12987	15500	1.19
1	6921	8500	1.23
2	9510	20600	2.17
3	11019	12900	1.17
4	23747	32500	1.37

Har seed me maximum grok *ke baad* aaya — bilkul usi pattern me jo baaki saare 6 predictors (L2 Norm, Dropout, Spectral, AGE, HTSR Alpha, Weight-PCA) dikha chuke hain.

17 Frozen Protocol Verdict

Official frozen protocol (`evaluate_frozen_protocol()`), sirf PRIMARY rule par (population omega maximum):

- $\text{lead_frac} \geq 0.8$ chahiye tha — mila 0.0000.
- $\rho \geq 0.9$ chahiye tha — mila 0.7000.
- $\text{mean_rel_lead} \in [0.05, 0.9]$ chahiye tha — mila -0.4254 .

Teeno thresholds fail hue. **VERDICT = FAIL**. Higher-MI tumhare benchmark ka 7-vaan predictor hai jo is stricter, leading-signal-wale protocol me fail hua hai.

Exploratory signals (scoring me nahi, sirf informational): zero-crossing epoch *bahut* jaldi aata hai (epoch 200–1600, training ke bilkul shuru me) — ye ek initialization artifact lagta hai, genuine signal nahi. Temporal-control maximum 4 me se 5 seeds me grok se pehle aata hai, par seeds ke across uski koi rank-correlation nahi hai grok epoch ke saath ($\rho = 0.00$) — yaani smooth hona kaafi nahi, usable predictor banne ke liye consistency bhi chahiye.

18 Implementation Ki Duvidha Aur Problems — Summary

Tumne specifically poochha tha ki is predictor me kya duvidha aur problems aayi. Ek jagah collect kar raha hoon:

- **Sabse badi conceptual duvidha (Section 14):** population-based adaptation lene ka faisla — kyunki paper ki literal time-series setting is project me exist hi nahi karti. Ye faisla implement karne se *pehle* discuss hua tha, aur isi duvidha ko seedha test karne ke liye temporal EXPLORATORY CONTROL add kiya gaya (sirf discuss karke chhod nahi diya gaya).
- **KSG ka $k = 4$ assumption:** paper is adapted logit-triplet setting ke liye k ki value explicitly nahi deta (unki original setting alag hai). $k = 4$ ek reasonable default assumption hai, par ye paper se directly nahi aata — flag karna zaroori tha.
- **Bug 1 (Section 11):** KSG ka strict-vs-inclusive neighbor-counting mismatch — ek subtle numerical bug jo crash nahi karta, bas systematically bias karta hai. Dense reference implementation ke against verify karna padha isse pakadne ke liye.
- **Bug 2 (Section 12):** test-loader seeding bug — silent correctness bug, bina seed ke test split training wale split se match nahi karta. Spectral/AGE predictors ke precedent ko dekh kar hi pakda gaya.
- **Compute cost:** naive (bina KSG tree-fix ke) implementation $\sim 19\text{s/checkpoint}$ leta — 2005 checkpoints (5×401) ke liye ~ 10.5 ghante, jo impractical tha. Fix ke baad $0.67\text{--}0.82\text{s/checkpoint}$, poora benchmark ~ 27 minute me complete hua.
- **Environment split:** is session ke device par torch install nahi ho paaya (CUDA shared-library dependency chain, non-GPU Mac par bhi) — isliye actual heavy compute ek alag cloud environment me chalaana pada, checkpoints compressed archives me transfer karke, aur result files wapas laa kar. Ye ek logistics problem thi, scientific nahi, par poora run complete karne ke liye zaroori thi.

19 Bada Picture: Thesis Ke Liye Matlab

Ye result bhi, WeightPCA ki tarah, “failure” nahi hai — khud ek finding hai. Pomarico et al. ka O-information idea apni original setting me valid ho sakta hai, par is project ka stricter test — jo ex-ante, leading signal maangta hai, sirf coincidence nahi — isko (aur ab tak 6 doosre published predictors ko) pass nahi karne deta.

Population aur temporal variants ka comparison (Section 16) ek extra layer deta hai jo doosre predictors ke paas nahi tha: ye seedha dikhata hai ki do alag reasonable adaptations bhi (population vs. literal-time-series) same jagah fail hoti hain, chahe unki raw curves qualitatively alag dikhti hon. Ye is finding ko thoda zyada robust banata hai — sirf ek specific adaptation-choice ki galti nahi lagti.

Agar aakhri bacha predictor (Commutator Defect) bhi isi tarah fail hota hai, to thesis ka actual contribution wahi rahega jo pehle establish ho chuka tha: koi bhi published predictor, is unified aur stricter protocol ke under, genuine ex-ante early-warning signal nahi deta.